



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



TFG del Grado en Ingeniería
Informática

SugAIr: aplicación móvil
proactiva para el control de
niveles de insulina en diabetes
Documentación Técnica



Presentado por Igor Arroyo Ortega
en Universidad de Burgos — 8 de julio de 2026
Tutor: Juan José Rodríguez Díez

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	iv
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	1
A.3. Estudio de viabilidad	5
Apéndice B Especificación de Requisitos	9
B.1. Introducción	9
B.2. Objetivos generales	9
B.3. Actores del sistema	10
B.4. Catálogo de requisitos	10
B.5. Especificación de requisitos	12
Apéndice C Especificación de diseño	15
C.1. Introducción	15
C.2. Diseño de datos	15
C.3. Diseño arquitectónico	16
C.4. Diseño procedimental	17
Apéndice D Documentación técnica de programación	19
D.1. Introducción	19
D.2. Estructura de directorios	19

D.3. Manual del programador	20
D.4. Compilación, instalación y ejecución del proyecto	21
D.5. Pruebas del sistema	23
Apéndice E Documentación de usuario	25
E.1. Introducción	25
E.2. Requisitos de usuarios	25
E.3. Instalación	27
E.4. Manual del usuario	28
Apéndice F Anexo de sostenibilización curricular	31
F.1. Introducción	31
F.2. Contextualización crítica y problemática global (SOS1)	31
F.3. Uso sostenible de recursos y eficiencia técnica (SOS2)	32
F.4. Participación comunitaria y equidad (SOS3)	33
F.5. Ética profesional y responsabilidad tecnológica (SOS4)	33
Bibliografía	35

Índice de figuras

A.1. Tablero Kanban de Igor	3
B.1. Diagrama de casos de uso de SugAIr.	12
C.1. Diagrama de arquitectura lógica del sistema SugAIr.	16
E.1. QR para abir con Expo Go	27
E.2. Pantalla de inicio	29
E.3. Botón clip de adjuntar imágenes	30

Índice de tablas

A.1. Resumen de costes económicos del proyecto.	6
A.2. Resumen de licencias de las herramientas y tecnologías utilizadas.	7
B.1. CU-01: Enviar consulta de texto.	13
B.2. CU-02: Consultar información nutricional mediante imagen.	14

Apéndice A

Plan de Proyecto Software

A.1. Introducción

Este apéndice explica la estrategia de gestión que ha guiado el ciclo de vida del desarrollo del asistente SugAIr. Para ello, se describe en primer lugar la organización temporal de las tareas y el seguimiento del progreso mediante metodologías ágiles. Asimismo, se incluye un análisis de la viabilidad del proyecto, evaluando tanto la inversión económica teórica necesaria para su ejecución como el marco normativo y legal que regula el despliegue y el tratamiento de los datos en la aplicación.

A.2. Planificación temporal

En el proyecto se han usado metodologías ágiles. Se han utilizado ciclos de desarrollo iterativos (*sprints*) de 3 semanas de duración a lo largo de un periodo de 4 meses. Para la gestión visual del flujo de trabajo, se ha utilizado un tablero Kanban alojado en GitLab, dividiendo los requisitos en tareas (*issues*).

Las tareas se definieron y asignaron a cada iteración con el objetivo de garantizar entregas funcionales e incrementales al final de cada *sprint* de 3 semanas.

A continuación, se detalla la evolución del desarrollo a través de los 5 *sprints* realizados:

Sprint 1: Configuración del entorno de desarrollo

Este primer ciclo abarcó la preparación de la infraestructura de hardware y software necesaria para soportar tanto el desarrollo móvil como el servidor backend.

- **Objetivos técnicos:** Instalación y configuración del entorno de desarrollo integrado (VSCode), el marco de trabajo móvil (React Native), el lenguaje de programación del servidor (Golang) y el resto de dependencias del ecosistema.

Sprint 2: Prueba de concepto y API REST base

El segundo *sprint* se enfocó en establecer el canal de comunicación inicial entre la aplicación móvil y el servidor.

- **Objetivos técnicos:** Investigación (*spike*) sobre la generación de código para APIs REST mediante especificaciones OpenAPI. Creación del proyecto base en React Native y desarrollo de un *endpoint* de comprobación de estado (*healthcheck*) en Golang, verificando la conexión mediante un botón en la *landing page* de la aplicación.
- **Documentación:** Redacción del título, resumen y *abstract* de la memoria.

Sprint 3: Interfaz flotante y motor de inferencia local

Esta iteración abordó dos de los requisitos fundamentales de la app: la ergonomía de la aplicación y la integración del motor de Inteligencia Artificial.

- **Objetivos técnicos:** Descarga, instalación y configuración del entorno Ollama, realizando una búsqueda del modelo de lenguaje local que mejor se ajustase a las especificaciones de hardware del equipo. En

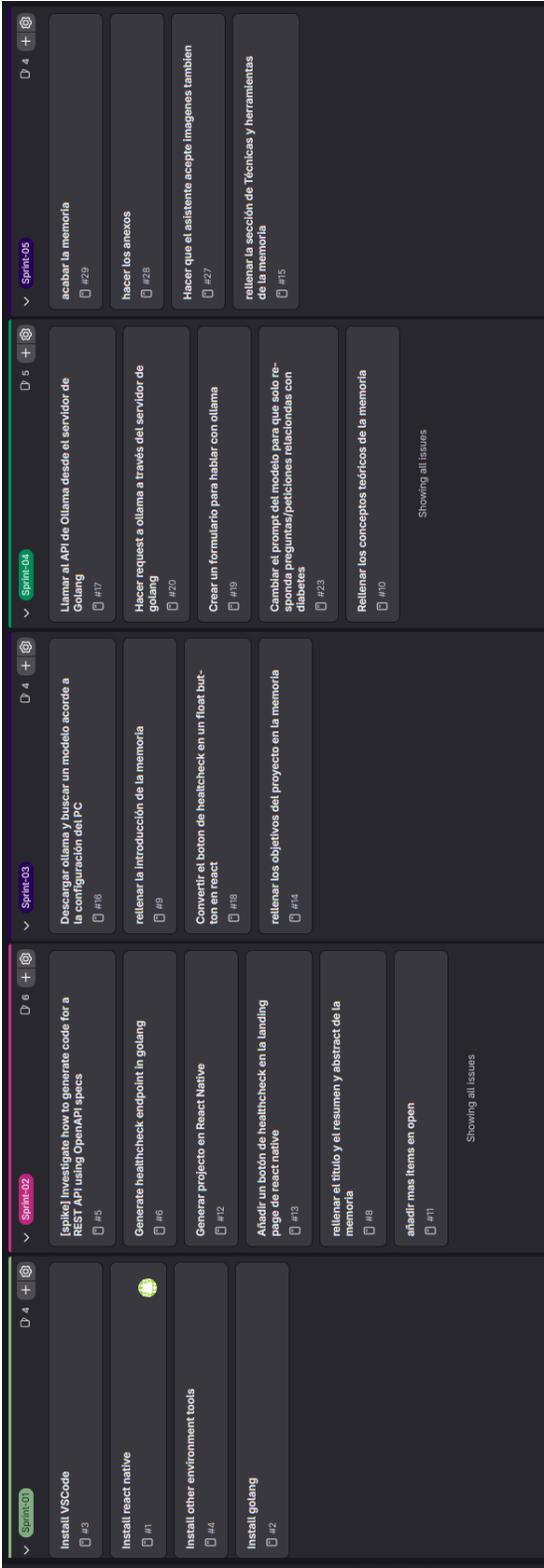


Figura A.1: Tablero Kanban de Igor

el *frontend*, se implementó la lógica para convertir el acceso a la herramienta en un botón flotante (*float button*). Pero en el diseño final se puso otro botón.

- **Documentación:** Redacción de la introducción y los objetivos del proyecto.

Sprint 4: Comunicación bidireccional y contexto clínico

En este ciclo se consolidó el flujo de datos completo, conectando la interfaz del usuario con el modelo de lenguaje.

- **Objetivos técnicos:** Desarrollo de un formulario de chat interactivo en React Native. En el *backend*, se implementaron las peticiones HTTP desde Golang hacia la API local de Ollama. Finalmente, se aplicaron técnicas de ingeniería de contexto (*prompt engineering*) para restringir el modelo, asegurando que las respuestas se limitaran exclusivamente a preguntas relacionadas con la diabetes.
- **Documentación:** Redacción del marco teórico y los conceptos de la memoria.

Sprint 5: Integración multimodal y cierre

El último *sprint* se dedicó a dotar al asistente de capacidades de visión artificial, superando las limitaciones previas, y a la finalización de los entregables académicos.

- **Objetivos técnicos:** Implementación del soporte para el procesamiento de imágenes. Se integró el modelo multimodal Llava y se adaptó el servidor Golang para gestionar y limpiar cadenas de imágenes en formato Base64, permitiendo al usuario adjuntar fotografías al chat (como gráficas de glucosa o etiquetas alimentarias).
- **Documentación:** Redacción del capítulo de técnicas y herramientas, desarrollo de los anexos y revisión final de la memoria.

A.3. Estudio de viabilidad

Viabilidad económica

El análisis de la viabilidad económica tiene como objetivo estimar los costes teóricos que supondría el desarrollo y puesta en marcha del proyecto SugAIr en un entorno empresarial real. Para realizar este cálculo, se han considerado los precios de mercado actuales en España y se ha supuesto un escenario de desarrollo de 4 meses de duración con una jornada laboral completa (8 horas diarias).

Costes de personal

El desarrollo ha sido realizado íntegramente por un perfil de Ingeniería Informática Junior. Se toma como referencia un salario bruto anual promedio para este perfil en España de 21.000,00 € [2]. Para obtener el coste real para la empresa, es necesario sumar al salario bruto los costes de la Seguridad Social (contingencias comunes, desempleo, formación, etc.), que se estiman aproximadamente en un 32 % del salario bruto.

Los cálculos para los 4 meses de desarrollo son los siguientes:

- Salario mensual bruto: $21,000,00 \text{ €} / 12 = 1,750,00 \text{ €} / \text{mes}$.
- Coste Seguridad Social (32 %): $1,750,00 \text{ €} \times 0,32 = 560,00 \text{ €} / \text{mes}$.
- Coste mensual total para la empresa: $1,750,00 \text{ €} + 560,00 \text{ €} = 2,310,00 \text{ €} / \text{mes}$.
- Coste total del proyecto (4 meses): $2,310,00 \text{ €} \times 4 = 9,240,00 \text{ €}$.

Costes de hardware

Para el desarrollo y la compilación de la Inteligencia Artificial local, se ha utilizado un equipo informático portátil (ASUS) con un precio de mercado estimado de 600,00 €. El coste imputable al proyecto se calcula mediante la amortización del equipo durante el tiempo de uso (4 meses), estimando una vida útil estándar de 4 años (48 meses).

- Amortización mensual: $600,00 \text{ €} / 48 = 12,50 \text{ €} / \text{mes}$.
- Coste total hardware (4 meses): $12,50 \text{ €} \times 4 = 50,00 \text{ €}$.

Costes de software e infraestructura

El proyecto SugAIr se ha desarrollado priorizando el uso de tecnologías de código abierto y herramientas gratuitas, lo que reduce drásticamente los costes de licencias. El *stack* tecnológico incluye Go, React Native, Ollama y Visual Studio Code, todos ellos gratuitos. El único coste de software imputable corresponde a una licencia comercial de Microsoft Windows 10, con un precio aproximado de 145,00 €, amortizada a 4 años ($3,02\text{€}/\text{mes} \times 4 = 12,08\text{€}$).

En cuanto a los costes de infraestructura, el diseño arquitectónico de SugAIr presenta una ventaja competitiva diferencial: **los costes de servidores en la nube y peticiones de API para inferencia de Inteligencia Artificial son de 0,00 €**. Al ejecutar los modelos de forma nativa a través del *runtime* local de Ollama, se elimina la dependencia económica de pasarelas de pago de terceros.

Resumen de costes

En la tabla A.1 se presenta el desglose final de los costes estimados para el desarrollo, incluyendo el Impuesto sobre el Valor Añadido (IVA) del 21 % vigente en España.

Concepto	Coste estimado (€)
Recursos Humanos (4 meses)	9.240,00
Hardware (Amortización)	50,00
Software (Sistema Operativo)	12,08
Infraestructura Cloud / Inferencia IA	0,00
Total Neto	9.302,08
IVA (21 %)	1.953,44
TOTAL PROYECTO	11.255,52

Tabla A.1: Resumen de costes económicos del proyecto.

Viabilidad legal

Para garantizar la viabilidad legal del proyecto, es fundamental analizar las licencias de los componentes tecnológicos utilizados y, dado el ámbito sanitario de la aplicación, el marco normativo sobre la privacidad de los datos.

Licencias de software de terceros

El sistema se ha construido empleando herramientas regidas en su gran mayoría por licencias permisivas de código abierto. En la tabla A.2 se presenta un resumen de las licencias de las principales tecnologías y librerías integradas en el proyecto, tanto en el servidor *backend* como en la aplicación móvil.

Herramienta / Librería	Uso en el proyecto	Licencia
Golang	Lenguaje base del servidor <i>backend</i>	BSD 3-Clause
Echo Framework	Enrutamiento y <i>middleware</i> REST	MIT License
React Native	<i>Framework</i> principal del <i>frontend</i>	MIT License
Expo (Router, Image Picker...)	Entorno de desarrollo y APIs nativas	MIT License
Axios	Cliente HTTP para peticiones de red	MIT License
Ollama	Motor de inferencia local de IA	MIT License
Llava (llava-diabetes)	Modelo multimodal base	LLaMA 2 Community License

Tabla A.2: Resumen de licencias de las herramientas y tecnologías utilizadas.

Como se observa, el ecosistema móvil y el servidor operan bajo licencias altamente permisivas como MIT y BSD de 3 cláusulas. En el caso del modelo multimodal Llava, al estar basado en la arquitectura LLaMA, se rige de forma heredada por la *LLaMA 2 Community License*, que permite su uso y modificación gratuitos para proyectos que no superen los 700 millones de usuarios activos mensuales, lo cual es plenamente compatible con el alcance de este trabajo.

Selección de licencia para el código propio

Considerando la naturaleza de las dependencias y el objetivo académico de este Trabajo Fin de Grado, se ha optado por liberar el código fuente de SugAIR bajo la licencia **MIT License**. Esta decisión fomenta la divulgación tecnológica, permitiendo usar, copiar, modificar y distribuir el *software*

sin restricciones, siempre que se mantenga el aviso de derechos de autor original. La documentación técnica se distribuye bajo la licencia **Creative Commons Attribution 4.0 International (CC BY 4.0)**.

Protección de datos y RGPD (Privacidad por Diseño)

Dado que SugAIr asiste en el control de la diabetes, la aplicación procesa datos considerados de categoría especial (datos relativos a la salud) según el artículo 9 del Reglamento General de Protección de Datos (RGPD) de la Unión Europea [4].

La viabilidad legal del proyecto ante este reglamento es excepcionalmente alta gracias a la aplicación del principio de **Privacidad por Diseño** (*Privacy by Design*). En lugar de recurrir a APIs de Inteligencia Artificial comerciales que procesan la información en servidores externos (lo cual exigiría complejos consentimientos explícitos, acuerdos de procesamiento de datos y auditorías de transferencias internacionales), la arquitectura de SugAIr delega todo el procesamiento multimodal al entorno local del usuario mediante Ollama.

Tanto las consultas de texto como las imágenes de las gráficas de medición y etiquetas alimentarias se procesan e infieren nativamente en el dispositivo, lo que garantiza que **ningún dato clínico o biométrico abandona la red local del paciente**. Al no existir almacenamiento ni transmisiones hacia la nube, el sistema cumple de forma inherente e impecable con las más estrictas exigencias legales sobre confidencialidad médica establecidas en el RGPD.

Apéndice B

Especificación de Requisitos

El presente apéndice recoge la documentación técnica detallada sobre la estructura interna y el flujo de ejecución de los componentes que integran el proyecto SugAIr.

B.1. Introducción

Este apéndice tiene como propósito definir y documentar detalladamente los requisitos funcionales y no funcionales que determinan el comportamiento del sistema SugAIr. Para ello, se establece un catálogo de requisitos y se especifican los casos de uso, describiendo las interacciones entre los actores y la propia plataforma. Esta especificación sirve como documento técnico para validar que el *software* construido satisface las necesidades y expectativas detectadas durante la fase de análisis inicial.

B.2. Objetivos generales

El desarrollo de SugAIr persigue proporcionar una herramienta tecnológica privada e inteligente para el apoyo diario a personas con diabetes. Los objetivos generales que guían la especificación de requisitos de este sistema son los siguientes:

- **Centralizar la gestión de la enfermedad:** Aunar en una única plataforma las funciones de educación, predicción y seguimiento de la diabetes mediante el uso de Inteligencia Artificial.

- **Desarrollar un agente especializado:** Proveer un asistente conversacional capaz de contestar preguntas única y exclusivamente sobre diabetes, actuando como un educador diabetológico virtual.
- **Integrar análisis de contexto visual:** Permitir el envío y procesamiento de imágenes (por ejemplo, etiquetas de alimentos) para extraer su valor nutricional, calcular raciones de hidratos de carbono y determinar su velocidad de absorción.
- **Garantizar la privacidad del usuario:** Procesar las interacciones y los datos médicos de forma local, evitando la exposición de información de salud en servidores de terceros.

B.3. Actores del sistema

Dado el enfoque de procesamiento local y la ausencia de una arquitectura tradicional con base de datos remota o perfiles de administración, el sistema interactúa de forma exclusiva con un único actor, definido formalmente a continuación:

- **Usuario:** Paciente, familiar o persona interesada que interactúa de forma directa con la interfaz de la aplicación móvil. Es el encargado de formular las consultas de texto, adjuntar las imágenes de las etiquetas nutricionales y recibir la retroalimentación del asistente virtual.

B.4. Catálogo de requisitos

Para el desarrollo de SugAIr, se han identificado y clasificado los requisitos en dos grandes grupos: funcionales (aquellos que describen los comportamientos y servicios que el sistema debe proveer) y no funcionales (aquellos que definen atributos de calidad, rendimiento y restricciones del sistema).

Requisitos Funcionales (RF)

- **RF-01:** El sistema debe permitir al usuario enviar consultas de texto mediante una interfaz de chat.
- **RF-02:** El sistema debe permitir al usuario adjuntar fotografías (ej. etiquetas nutricionales) desde la galería de su dispositivo.

- **RF-03:** El sistema debe recibir, procesar y mostrar en pantalla las respuestas generadas por el motor de IA local.
- **RF-04:** El sistema debe restringir las temáticas de conversación, rechazando o evadiendo aquellas consultas que no estén relacionadas con el ámbito de la diabetes o la nutrición.

Requisitos No Funcionales (RNF)

- **RNF-01 (Privacidad Local):** Todo el procesamiento multimodal y de lenguaje debe ejecutarse en el entorno local del usuario, sin enviar información clínica a servidores externos.
- **RNF-02 (Rendimiento):** El sistema debe ofrecer tiempos de respuesta razonables para un entorno móvil, idealmente devolviendo las respuestas de texto en menos de 15 segundos.
- **RNF-03 (Usabilidad):** La interfaz de usuario debe ser intuitiva, accesible y seguir los estándares de diseño de aplicaciones de mensajería conversacional.

B.5. Especificación de requisitos

A continuación, se detallan los Casos de Uso (CU) del sistema, derivados del catálogo de requisitos funcionales. En la figura B.1 se presenta el diagrama general de casos de uso (UML), resumiendo visualmente las interacciones del actor con el sistema.

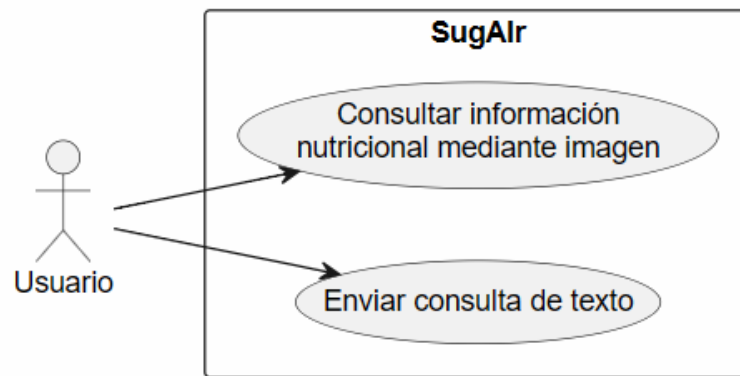


Figura B.1: Diagrama de casos de uso de SugAIr.

Cada caso de uso describe una interacción específica entre el Usuario y la aplicación SugAIr, definiendo sus precondiciones, flujo de acciones y posibles excepciones.

CU-01	Enviar consulta de texto
Versión	1.0
Autor	Igor Arroyo Ortega
Requisitos asociados	RF-01, RF-03, RF-04
Descripción	El usuario formula una pregunta relacionada con la diabetes o nutrición mediante texto y el sistema devuelve una respuesta generada por la IA.
Precondición	• La aplicación debe estar abierta. • El servidor backend (Go) y Ollama deben estar en ejecución local.
Acciones	1. El usuario pulsa sobre la caja de texto en la interfaz del chat. 2. El usuario escribe su consulta y pulsa el botón de enviar. 3. La aplicación bloquea el botón de envío y muestra un indicador de carga (<i>loading</i>). 4. El sistema empaqueta la petición en formato JSON y la envía al servidor backend. 5. El backend consulta al modelo <code>llava-diabetes</code>. 6. El sistema recibe la respuesta, oculta el indicador de carga y la renderiza en el chat.
Postcondición	La conversación queda actualizada en la pantalla con la nueva respuesta del asistente.
Excepciones	• El servidor local está apagado (La app muestra un mensaje de error). • Temática no sanitaria (La IA responde recordando sus limitaciones).
Importancia	Alta

Tabla B.1: CU-01: Enviar consulta de texto.

CU-02	Consultar información nutricional mediante imagen
Versión	1.0
Autor	Igor Arroyo Ortega
Requisitos asociados	RF-02, RF-03, RF-04
Descripción	El usuario adjunta una fotografía (ej. etiqueta de un alimento) para que la IA analice la imagen y extraiga su valor nutricional.
Precondición	• Permisos de galería concedidos en el dispositivo. • El servidor backend y Ollama están en ejecución.
Acciones	1. El usuario pulsa el botón de adjuntar archivo en el chat. 2. Selecciona una fotografía desde la galería del dispositivo. 3. La aplicación comprime la imagen y extrae directamente su codificación en formato Base64 puro. 4. El usuario pulsa enviar (pudiendo añadir texto opcional a la imagen). 5. El servidor backend recibe la petición y reenvía la cadena Base64 directamente al modelo multi-modal. 6. El sistema recibe y renderiza la respuesta de la IA en la pantalla.
Postcondición	La conversación refleja el análisis nutricional devuelto por el asistente basándose en la imagen enviada.
Excepciones	• Permisos de galería denegados por el sistema operativo. • Error del servidor (Se captura el estado HTTP y se muestra en pantalla).
Importancia	Alta

Tabla B.2: CU-02: Consultar información nutricional mediante imagen.

Apéndice C

Especificación de diseño

C.1. Introducción

Este apéndice detalla la arquitectura interna del sistema SugAIr y las decisiones de diseño tomadas durante su desarrollo. El objetivo es proporcionar una visión técnica clara sobre cómo se estructuran e interactúan los distintos componentes de *software* que conforman la plataforma. Al tratarse de un sistema centrado en la Inteligencia Artificial y la privacidad, la arquitectura difiere de las aplicaciones móviles tradicionales basadas en la nube, adoptando un enfoque de procesamiento local y sin persistencia de datos tradicional.

C.2. Diseño de datos

Dada la naturaleza del proyecto y el requisito fundamental de Privacidad por Diseño, el sistema SugAIr carece de una base de datos persistente (como SQL o NoSQL). No se almacena historial de conversaciones, perfiles de usuario ni telemetría. El diseño de datos se centra exclusivamente en las estructuras de intercambio de información en memoria, utilizando el formato JSON para la comunicación entre capas.

Para estandarizar este intercambio, la API REST se ha diseñado siguiendo la especificación OpenAPI (versión 3.0.3). Las estructuras de datos principales son las siguientes:

- **Objeto de Petición (ChatRequest):** Estructura enviada desde la aplicación móvil al servidor. Contiene dos propiedades de tipo tex-

to (*string*): **content** (el mensaje del usuario) e **image** (la fotografía codificada en Base64, de carácter opcional).

- **Objeto de Respuesta (**ChatResponse**):** Estructura devuelta por el servidor a la aplicación móvil. Contiene una única propiedad de tipo texto llamada **message**, que incluye la respuesta generada por la IA.
- **Objeto de Inferencia (**OllamaRequest**):** Estructura interna generada por el servidor backend para comunicarse con el motor local. Incluye el nombre del modelo (**model**: *llava-diabetes*), el parámetro **stream** (configurado en *false* para respuestas síncronas) y la matriz de imágenes.

C.3. Diseño arquitectónico

El sistema SugAIr se ha diseñado siguiendo un patrón arquitectónico Cliente-Servidor estructurado en **tres capas lógicas**. Esta separación de responsabilidades facilita el escalado de componentes de *software* independientes y garantiza que el pesado procesamiento de los modelos de IA no sature la interfaz. En la figura C.1 se ilustra el diagrama de componentes lógicos de la plataforma.

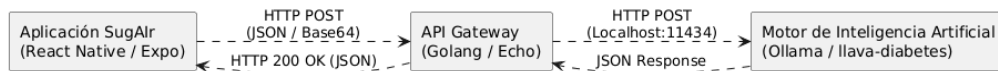


Figura C.1: Diagrama de arquitectura lógica del sistema SugAIr.

Capa de Presentación (Frontend)

Constituye el punto de interacción directo con el usuario final. Está implementada como una aplicación móvil multiplataforma desarrollada con **React Native** y el entorno de herramientas de **Expo**. Es responsable de renderizar la interfaz de usuario, gestionar el estado dinámico de la conversación, acceder a las APIs nativas del dispositivo (galería de imágenes) y aplicar la compresión inicial de las fotografías.

Capa Lógica (API Gateway)

Actúa como un puente seguro y un orquestador de peticiones. Está desarrollada en **Golang** haciendo uso del *framework* de enrutamiento rápido

Echo. Recibe las peticiones de la aplicación móvil en el puerto 8080 y gestiona el enrutamiento bidireccional hacia el motor local, transformando los datos al formato específico requerido y controlando los tiempos de espera (*timeouts*).

Capa de Inferencia

SugAIr emplea un motor de inferencia local impulsado por **Ollama**. Es el encargado de alojar en memoria y ejecutar el modelo de lenguaje multimodal **llava-diabetes**. Decodifica las imágenes en Base64, analiza el texto del usuario aplicando la comprensión del contexto clínico y genera la respuesta final.

C.4. Diseño procedimental

El diseño procedimental define la lógica interna y el flujo algorítmico que sigue el sistema para resolver una tarea. En SugAIr, el procedimiento principal es el ciclo de envío y recepción de un mensaje.

El algoritmo implementado en la aplicación móvil (`index.tsx`) y en el servidor (`main.go`) sigue el siguiente flujo de ejecución síncrono:

1. **Validación de entrada:** La interfaz bloquea el envío si el campo de texto está vacío y no hay una imagen adjunta. Si hay imagen, se extrae de la galería y se comprime directamente a Base64 puro.
2. **Bloqueo de interfaz:** Se activa el estado de carga (*loading*) mostrando un indicador visual (`ActivityIndicator`) y deshabilitando temporalmente el botón de envío.
3. **Petición de red:** El cliente formatea la petición HTTP POST hacia `/api/v1/chat` utilizando la librería *Axios* y queda a la espera.
4. **Orquestación (*Backend*):** El servidor en Go recibe la petición, extrae el cuerpo JSON y construye una nueva petición dirigida a la API interna de Ollama (`localhost:11434`).
5. **Inferencia:** El modelo **llava-diabetes** procesa la información y devuelve la respuesta.
6. **Resolución y renderizado:** El servidor Go recibe la respuesta de Ollama, comprueba que el código HTTP sea 200 (OK) y se la devuelve

al cliente móvil. La interfaz desactiva el estado de carga, inyecta la nueva respuesta en el contenedor de mensajes de la pantalla y restablece los controles. En caso de error de red o estado HTTP anómalo, la lógica captura la excepción e imprime un mensaje de error visualizado en color rojo.

Apéndice D

Documentación técnica de programación

El presente apéndice constituye la guía técnica oficial para el mantenimiento, despliegue y evaluación empírica del código fuente del sistema SugAIr, sirviendo como marco de referencia para futuros desarrolladores o auditores del proyecto.

D.1. Introducción

La documentación técnica es un pilar fundamental en la ingeniería del *software* para garantizar la transferibilidad y sostenibilidad de un sistema. En esta sección se abordan los aspectos prácticos de la construcción de SugAIr, organizando de manera exhaustiva la disposición física de los ficheros en el entorno de desarrollo, las pautas operativas para futuros programadores, el procedimiento exacto de despliegue local y las métricas obtenidas tras la ejecución de las pruebas automatizadas del servidor.

D.2. Estructura de directorios

Para facilitar la comprensión global de la plataforma, el repositorio de SugAIr se ha estructurado dividiendo limpiamente los módulos lógicos correspondientes al servidor en Go, la interfaz móvil en React Native y la parametrización de la Inteligencia Artificial. A continuación, se desglosa la jerarquía de directorios definitiva del entorno de desarrollo:

- **/go/ (Capa de lógica y enrutamiento - Backend)**
 - `main.go`: Punto de entrada del backend que inicializa el *framework* Echo y gestiona las peticiones de red.
 - `openapi.yaml`: Contrato de diseño de la interfaz API REST siguiendo el estándar internacional OpenAPI 3.0.
 - `api.gen.go`: Código del servidor generado automáticamente para asegurar la coherencia estricta con el contrato de la API.
 - `main_test.go`: Batería de pruebas automatizadas unitarias y de integración para los controladores del servidor.
 - `go.mod` y `go.sum`: Ficheros destinados al control, registro y bloqueo de versiones de las dependencias de Go.
- **/ollama/multimodal/ (Configuración del motor de inferencia)**
 - `modelfile`: Fichero instructivo de Ollama que especifica el uso del modelo base Llava y las directrices del *System Prompt*.
- **/React_native/front-end/ (Capa de presentación móvil - Frontend)**
 - `app/`: Núcleo conversacional bajo el sistema *Expo Router* que contiene las vistas de navegación (`(tabs)/`), la lógica principal del chat (`index.tsx`) y la configuración estructural (`_layout.tsx`).
 - `components/` y `hooks/`: Directorios destinados a la modularización de elementos visuales aislados de React y lógica asíncrona reusable.
 - `assets/`: Almacén de recursos gráficos, imágenes y elementos estáticos de marca de la aplicación móvil.
 - `package.json`: Declaración de dependencias del entorno Node.js y configuración de los *scripts* de ejecución de Expo.

D.3. Manual del programador

La extensión funcional o modificación del ecosistema SugAIr requiere que cualquier intervención futura respete escrupulosamente los patrones de diseño aplicados durante su concepción. El desarrollo de nuevas características debe regirse por los siguientes principios técnicos de mantenimiento:

1. **Gestión de flujos en la capa móvil:** Toda integración visual en la aplicación React Native debe implementarse dentro del directorio `app/`. Al utilizar enrutamiento basado en archivos (*Expo Router*), la creación de nuevas pantallas requiere únicamente añadir un fichero con extensión `.tsx`. Es mandatorio encapsular las consultas HTTP asíncronas haciendo uso de estados reactivos para gobernar correctamente el bloqueo de botones y los componentes de carga activa (`ActivityIndicator`).
2. **Ciclo de modificación de la API de enrutamiento:** Para alterar o ampliar la comunicación del backend, se prohíbe explícitamente modificar de forma directa el fichero `api.gen.go`. El flujo de trabajo correcto es actualizar primero el diseño contractual en `openapi.yaml`, ejecutar la herramienta de generación automatizada para actualizar los tipos de datos en Go y, finalmente, implementar la lógica operativa dentro de los controladores de `main.go`.
3. **Alteración heurística del asistente algorítmico:** Los cambios asociados a la especialización del educador diabetológico virtual no se gestionan mediante código tradicional. Cualquier restricción temática o cambio en el formato de salida debe realizarse modificando la directiva `SYSTEM` en el fichero `modelfile` de la carpeta multimodal, requiriendo su posterior ensamblado en el motor Ollama.

D.4. Compilación, instalación y ejecución del proyecto

El despliegue local de la plataforma en un entorno de desarrollo requiere que la máquina anfitriona disponga de los entornos de ejecución de Golang, Node.js y el servicio Ollama previamente instalados. El ciclo completo para inicializar los tres componentes esenciales de la arquitectura se compone de los siguientes procedimientos secuenciales:

1. Construcción del modelo de inferencia

Antes de arrancar los servicios lógicos, es indispensable disponer del entorno de ejecución y construir el modelo especializado. En primer lugar, se debe descargar e instalar el motor de Inteligencia Artificial en el equipo anfitrión desde su sitio web oficial (<https://ollama.com/>).

Una vez instalado, es necesario descargar el modelo base multimodal sobre el cual se asienta el sistema. Desde la consola de comandos, se ejecuta la siguiente instrucción:

```
ollama pull llava
```

A continuación, se navega hasta el directorio que contiene la configuración del proyecto y se compila el modelo personalizado (que incluye el *System Prompt* médico) ejecutando las siguientes instrucciones:

```
cd ollama  
ollama create llava-diabetes -f modelfile
```

2. Inicialización del servidor backend

Una vez disponible el modelo en la memoria local, se abre una nueva ventana en la terminal y se arranca el puente de enrutamiento en Go:

```
cd go  
go mod tidy  
go run .
```

El servidor confirmará su activación mostrando en consola que se encuentra a la escucha de peticiones en el puerto 8080.

3. Lanzamiento de la aplicación móvil cliente

Finalmente, se procede a compilar el cliente móvil interactivo mediante el gestor de paquetes y el empaquetador de Expo:

```
cd React_native/front-end  
npm install  
npx expo start
```

La terminal desplegará un código QR dinámico. El desarrollador podrá escanear dicho código a través de la herramienta móvil *Expo Go* para sincronizar y ejecutar la aplicación en tiempo real en un terminal físico. También le puede dar a la tecla w para verlo en versión web.

D.5. Pruebas del sistema

Para validar la fiabilidad y resiliencia del *software* construido, se ha adoptado una estrategia de pruebas combinada, integrando análisis automatizados en el backend y verificaciones de rendimiento e interfaz en el entorno cliente.

Métricas de cobertura automatizada (Backend)

Haciendo uso de las herramientas de inspección del compilador de Go, se ha diseñado una batería de pruebas unitarias y de integración en el fichero `main_test.go`. Estas pruebas evalúan de forma controlada el comportamiento de la API frente a escenarios críticos, tales como la validación del estado del sistema (*healthcheck*) o la inyección deliberada de cargas JSON corruptas.

Mediante el uso de servidores web simulados en memoria (`httptest.NewServer`), se ha logrado aislar la lógica del servidor sin depender del estado externo del motor Ollama. Al ejecutar las pruebas a través de la instrucción oficial `go test -v -cover`, el sistema arroja una métrica de cobertura del 47,9 % de las sentencias globales.

Este porcentaje representa una cobertura robusta de los flujos principales de ejecución (*Happy Paths*) y de los mecanismos de validación de entrada (*Bad Request*). La cobertura restante corresponde de forma deliberada al bloque de inicialización y bloqueo de la función `main()`, así como a las ramificaciones secundarias de captura de excepciones de red interna (tiempos de espera agotados o caídas del motor IA), cuya verificación requeriría técnicas de inyección de fallos más complejas que escapan al alcance de estas pruebas unitarias.

Validación de rendimiento conversacional e interfaz

De manera complementaria a los tests automatizados, se realizaron pruebas manuales en el entorno móvil para medir la robustez de la interfaz frente a escenarios de error, tales como cortes de conexión (servidor backend desconectado) y denegación de permisos del sistema operativo para acceder a la galería fotográfica.

Asimismo, se explotaron empíricamente los resultados de rendimiento del sistema. Estas pruebas de carga demostraron la viabilidad computacional de la inferencia local multimodal, confirmando que la compresión previa aplicada a las imágenes desde React Native disminuye sustancialmente el

peso del *payload*, permitiendo que los tiempos de procesamiento de Ollama se mantengan estables y dentro de un rango aceptable para una interacción conversacional.

Apéndice *E*

Documentación de usuario

El presente apéndice constituye el manual de usuario final del sistema SugAIr. Su propósito principal es guiar al paciente o usuario a través de la interfaz móvil, explicando los requisitos técnicos necesarios para ejecutar el prototipo, el proceso de instalación en un entorno de pruebas y las instrucciones de uso para interactuar correctamente con el asistente virtual de nutrición.

E.1. Introducción

SugAIr se ha diseñado con un enfoque centrado en la accesibilidad y la simplicidad, adoptando el paradigma visual de las aplicaciones de mensajería instantánea tradicionales. Al ocultar la complejidad técnica del procesamiento de Inteligencia Artificial y la comunicación en red, el usuario final solo necesita interactuar con una interfaz de chat intuitiva para obtener asistencia nutricional especializada en diabetes.

E.2. Requisitos de usuarios

Para garantizar el correcto funcionamiento del prototipo en su estado actual de desarrollo, el usuario debe disponer de un entorno que cumpla con los siguientes requisitos técnicos y de permisos:

- **Hardware:** Un dispositivo móvil inteligente (*smartphone*) con sistema operativo Android o iOS.

- **Conectividad:** Conexión activa a la misma red de área local (Wi-Fi) en la que se encuentra desplegado el servidor principal del sistema.
- **Software de terceros:** La aplicación gratuita *Expo Go* instalada en el dispositivo móvil.
- **Permisos del sistema:** Concesión explícita de permisos de acceso a la galería multimedia del dispositivo para permitir la selección y envío de etiquetas nutricionales o fotografías de alimentos.



Figura E.1: QR para abrir con Expo Go

E.3. Instalación

Al tratarse de una prueba de concepto enmarcada en un Trabajo de Fin de Grado, la aplicación no se encuentra distribuida mediante las tiendas de aplicaciones comerciales (Google Play Store o Apple App Store). El despliegue en el terminal del usuario se realiza mediante sincronización local siguiendo estos pasos:

1. Descargar e instalar la aplicación **Expo Go** desde la tienda oficial correspondiente al sistema operativo del dispositivo móvil.
2. Asegurarse de que el dispositivo móvil está conectado a la misma red Wi-Fi que el dispositivo que aloja el servidor de SugAir.
3. Solicitar al administrador del sistema (el desarrollador) que inicie el entorno de ejecución, el cual generará un código QR en la pantalla del ordenador.
4. Abrir la aplicación Expo Go en el dispositivo móvil y seleccionar la opción *Scan QR Code*.
5. Enfocar el código QR. La aplicación descargará el paquete de JavaScript, compilará la interfaz y ejecutará SugAir automáticamente de forma nativa.

Se podría ejecutar todo en el móvil, pero tendría que ser un dispositivo potente y con bastante RAM. Sería instalar Termux o algún terminal para Android, e instalar Go y Ollama para ejecutar los comandos de la guía del manual del programador

E.4. Manual del usuario

Una vez ejecutada la aplicación, el usuario interactuará con una única pantalla principal que centraliza todas las funcionalidades del asistente médico. El flujo de uso básico se divide en las siguientes acciones:

1. Interfaz principal y mensajería de texto

Al abrir SugAIr, la pantalla mostrará un lienzo en blanco o un mensaje de bienvenida como la de la imagen E.2. En la parte inferior, anclada sobre el teclado virtual, se encuentra la barra de interacción. Para realizar una consulta escrita estándar:

- Pulsar sobre el cajón de texto que indica *.Escribe tu mensaje...*.
- Redactar la consulta deseada.
- Pulsar el botón azul con el icono de enviar (flecha) situado en el extremo derecho.

2. Análisis visual de alimentos, graficas de actividad física e información nutricional

La característica principal del sistema es su capacidad multimodal. Para que la IA analice una fotografía de una etiqueta nutricional o un alimento o una gráfica de actividad física:

- Pulsar el botón con el icono de un **clip** E.3 (adjuntar), situado a la izquierda de la barra de texto.
- El sistema operativo abrirá la galería de imágenes. El usuario debe seleccionar la fotografía deseada.
- Una vez seleccionada, aparecerá una pequeña imagen de previsualización justo encima de la barra de texto.
- Si el usuario se ha equivocado de imagen, puede descartarla pulsando el icono circular rojo con una cruz situado en la esquina de la miniatura.
- Para enviar la imagen a análisis, se debe pulsar el botón azul de enviar. Opcionalmente, se puede añadir texto explicativo en el cajón de mensajería antes de realizar el envío.

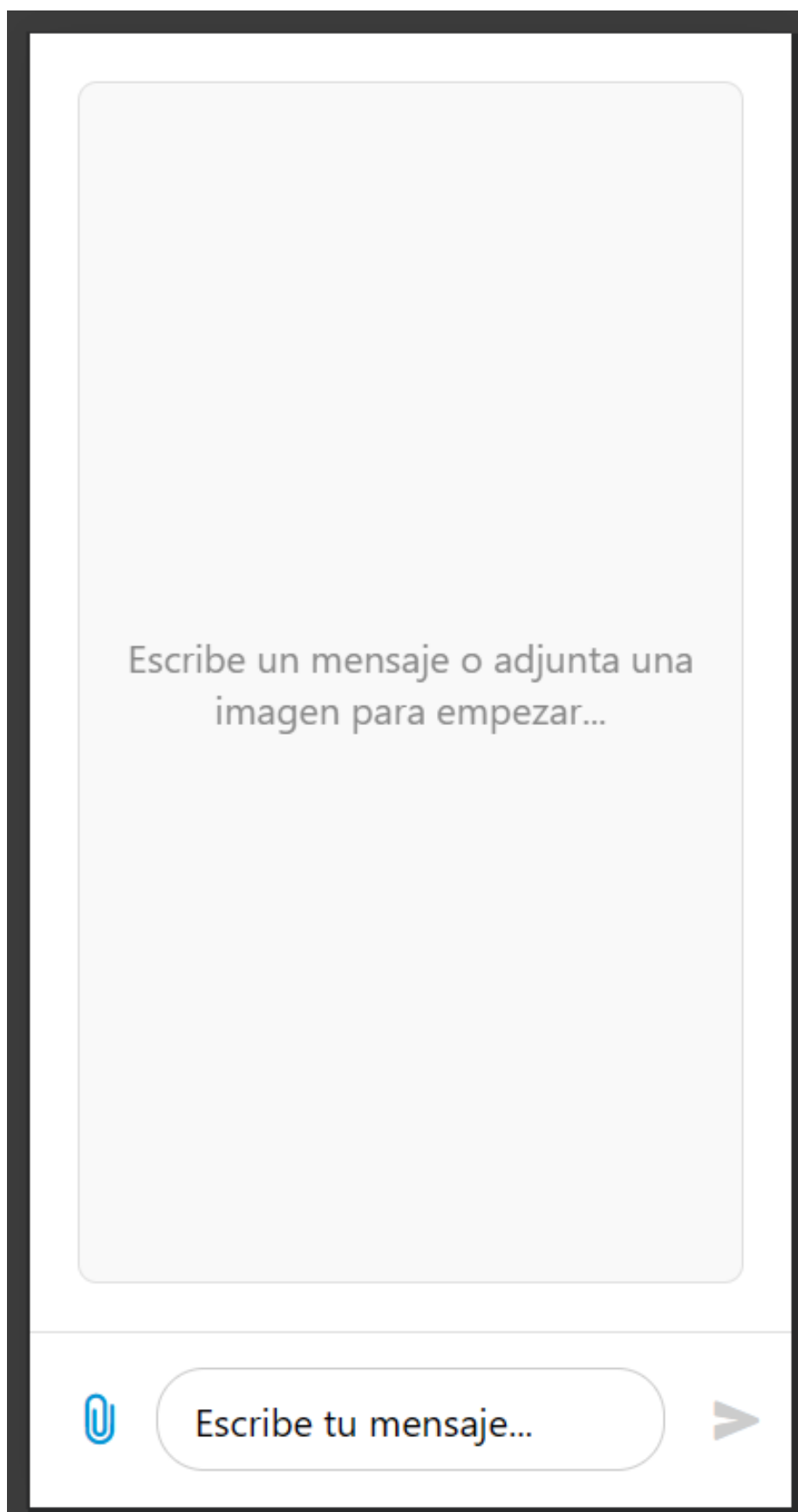


Figura E.2: Pantalla de inicio

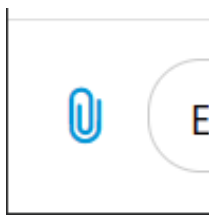


Figura E.3: Botón clip de adjuntar imágenes

3. Espera y recepción de resultados

Dado que el análisis de imágenes mediante Inteligencia Artificial requiere capacidad de cómputo, el procesamiento no es instantáneo.

- Tras pulsar el botón de envío, el cajón de texto y la imagen se limpiarán, y los botones quedarán temporalmente bloqueados para evitar envíos duplicados.
- En el centro de la pantalla aparecerá un indicador de carga circular animado de color azul, indicando que el sistema está procesando la solicitud.
- Una vez finalizado el cálculo, el indicador desaparecerá y el mensaje con el consejo diabetológico se mostrará por pantalla.

4. Gestión de errores

Si ocurre un problema de conexión (por ejemplo, pérdida de señal Wi-Fi o caída del servidor principal), el sistema protegerá la experiencia del usuario mostrando un mensaje de error en color rojo en la pantalla principal (por ejemplo: *Error de red* o *Status: 500*). Ante esta situación, el usuario deberá verificar su conexión y volver a intentar el envío.

Anexo de sostenibilización curricular

F.1. Introducción

El desarrollo del Trabajo de Fin de Grado no solo representa la culminación de la adquisición de competencias técnicas en el ámbito de la ingeniería de *software*, sino que exige una reflexión profunda sobre el impacto que la tecnología generada ejerce sobre la sociedad y el entorno natural. Siguiendo las directrices de la CRUE para la introducción de la sostenibilidad en el currículum universitario [1], el diseño, la programación y el despliegue del ecosistema SugAIr han integrado de forma transversal las cuatro competencias fundamentales de sostenibilidad. A lo largo de este anexo se detalla cómo el proyecto aborda la contextualización crítica de la salud, el uso eficiente y responsable de los recursos computacionales, la participación de la comunidad y la ineludible ética profesional en el manejo de datos médicos.

F.2. Contextualización crítica y problemática global (SOS1)

La primera competencia de sostenibilidad requiere establecer interrelaciones entre los problemas sociales, económicos y ambientales. SugAIr nace como respuesta tecnológica directa a una crisis sociosanitaria de carácter global: el incremento progresivo de la prevalencia de la diabetes y la falta de educación nutricional continua en la población [3].

Desde la dimensión social, la aplicación democratiza el acceso al conocimiento médico-nutricional. La capacidad del modelo base para analizar visualmente etiquetas de alimentos y resumir macronutrientes elimina barreras cognitivas y de comprensión lectora. De este modo, se empodera a los pacientes para que tomen decisiones informadas sobre su dieta diaria de forma autónoma, mitigando la dependencia exclusiva de las escasas visitas a los especialistas en endocrinología.

En la vertiente económica, la autogestión eficaz de la ingesta de hidratos de carbono impacta de forma directa en la viabilidad de los sistemas de salud pública. La prevención de los picos glucémicos reduce drásticamente la incidencia de comorbilidades graves a largo plazo. Esta prevención se traduce en un menor gasto en ingresos hospitalarios, urgencias y farmacología. Además, la concepción del prototipo como una herramienta gratuita y de fácil acceso contribuye a reducir la brecha digital y económica.

F.3. Uso sostenible de recursos y eficiencia técnica (SOS2)

La segunda directriz se centra en la utilización sostenible de los recursos. En el campo del desarrollo de *software*, la sostenibilidad ambiental se materializa a través de la eficiencia algorítmica y la reducción de la huella energética de la infraestructura computacional.

El diseño arquitectónico de SugAIr se ha regido por la ligereza y el alto rendimiento. El uso del lenguaje Golang para la construcción del servidor intermedio garantiza un consumo ínfimo de memoria RAM y de ciclos de procesador, situándose como uno de los lenguajes compilados más eficientes energéticamente [5], lo que reduce el consumo eléctrico del equipo anfitrión.

De manera análoga, la decisión de utilizar el motor local Ollama para la Inteligencia Artificial supone una estrategia de descentralización sostenible. Al ejecutar las inferencias de forma local, se elimina la dependencia de macrocentros de datos masivos alojados en la nube, mitigando la huella de carbono asociada al entrenamiento y tráfico de red masivo de los grandes modelos de lenguaje [6]. A nivel de cliente, la aplicación en React Native ejecuta rutinas de compresión de imágenes previas a la transmisión HTTP, minimizando el consumo de ancho de banda y alargando la vida útil de la batería del terminal.

F.4. Participación comunitaria y equidad (SOS3)

La tercera competencia persigue promover procesos que fomenten el desarrollo sostenible de la comunidad. SugAIr no es una simple base de datos consultiva, sino un agente educador activo diseñado para mejorar la alfabetización nutricional de la comunidad diabética, otorgando al usuario un rol proactivo.

Para asegurar que este beneficio trascienda, el proyecto se asienta firmemente en la filosofía de código abierto (*Open Source*). Todas las capas tecnológicas elegidas (Expo, Golang, Ollama y el modelo Llava) son de libre acceso. Esto asegura que la comunidad global de desarrolladores e investigadores puedan auditar, bifurcar y expandir el sistema sin restricciones por licencias privativas, garantizando la perdurabilidad del conocimiento.

F.5. Ética profesional y responsabilidad tecnológica (SOS4)

Por último, la competencia ética cobra una dimensión crítica al integrar modelos de Inteligencia Artificial en el ámbito sanitario. La implementación de SugAIr asume una responsabilidad profesional innegociable centrada en dos pilares: la protección de la información y la limitación clínica, alineándose con las directrices internacionales sobre la ética de la IA en la salud [7].

Al basarse en modelos que pueden ser ejecutados en arquitecturas propias, se respeta la soberanía de los datos del paciente. Las fotografías y consultas privadas no son recopiladas ni comercializadas por corporaciones de terceros para el reentrenamiento indiscriminado de algoritmos, preservando así su derecho a la privacidad.

Por otra parte, se ha actuado con rigor ético para evitar la negligencia algorítmica. Mediante un *System Prompt* restrictivo, se ha limitado el radio de acción del asistente, obligándolo a declinar cualquier solicitud de diagnóstico o prescripción de fármacos. La herramienta se define éticamente como un apoyo dietético no vinculante, protegiendo al usuario de las alucinaciones inherentes a la IA generativa y garantizando que la tecnología actúe como un complemento seguro y nunca sustituya el criterio clínico humano.

Bibliografía

- [1] Comisión Sectorial CRUE-Sostenibilidad. Directrices para la introducción de la sostenibilidad en el curriculum. Technical report, Conferencia de Rectores de las Universidades Españolas (CRUE), 2012.
- [2] Glassdoor. Sueldo: Programador junior. https://www.glassdoor.es/Sueldos/programador-junior-sueldo-SRCH_K00,18.htm. [Fecha de consulta: 10 de julio de 2026].
- [3] International Diabetes Federation. *IDF Diabetes Atlas*. International Diabetes Federation, Brussels, Belgium, 10th edition, 2021.
- [4] Parlamento Europeo y Consejo de la Unión Europea. Reglamento (ue) 2016/679 (reglamento general de protección de datos). <https://eur-lex.europa.eu/eli/reg/2016/679/oj>. Artículo 9: Tratamiento de categorías especiales de datos personales.
- [5] Rui Pereira, Marco Couto, Francisco Ribeiro, Roberto Rua, Jácome Cunha, João Paulo Fernandes, and João Saraiva. Energy efficiency across programming languages: how do energy, time, and memory relate? *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering*, pages 256–267, 2017.
- [6] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in nlp. *arXiv preprint arXiv:1906.02243*, 2019.
- [7] World Health Organization. *Ethics and governance of artificial intelligence for health: WHO guidance*. World Health Organization, Geneva, 2021.